Pattern Recognition Project Report

[Team number 10]

Joo Minsuh(2391029), Choi Dabin(2391031),

Choi Yeonseo(2391032), Jeon Jiyoon(2392029)

-Ewha Womans University-

Department of Artificial Intelligence

Table of Contents

I. Data Exploration

II. Data Preprocessing

III. Model Training and Validation

IV. Interpretation and Analysis

V. References

I. Data Exploration

1. Histogram

A goal of our project is to successfully predict whether a particular client will subscribe to a bank term deposit from the given data. If a target value 'y' has a value of '1', it means that a client will subscribe to a deposit. In contrast, if it has a value of '0', a client will not subscribe to a deposit. When we visualized the counts of class 0 and 1 of 'y', the counts for class 0 were pretty dominant, showing almost 8 times larger than class 1.

Before preprocessing data and training models, we have to preferentially explore the given data. Let's look at each variable in the training data set. 'job', 'marital', 'education', 'default', 'housing', 'loan', 'contact', 'month', 'day of week', 'poutcome', 'y'(target value) are all categorical variables. The rest are all numerical variables.

Through visualization we are going to examine the meaning of each variable and its relationship with the target value.

1.1 id

This variable represents the ID number of clients. It is not related to increasing the score of models but only used to distinguish the client. After the final prediction, we will concatenate 'id' as indexes of the result of prediction.

1.2 age

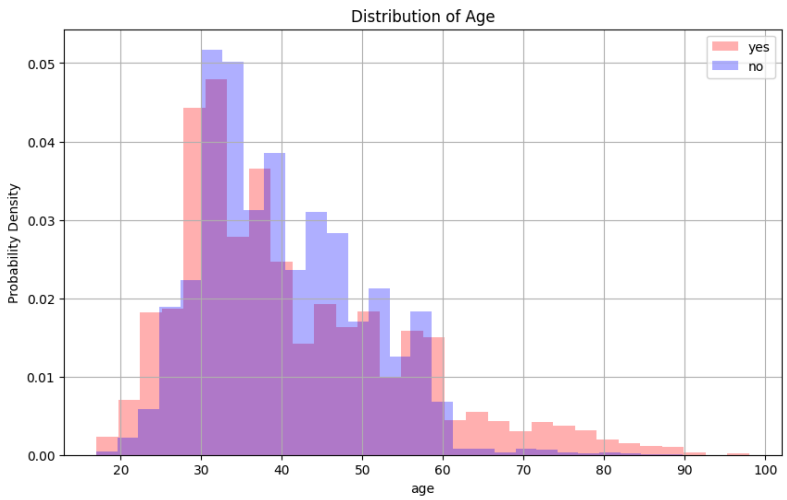


Figure 1: Histogram of 'age'

Figure 1 shows a result of visualizing the 'age' of subscribers and non-subscribers. The absolute number of data is most distributed in 30s~40s. Also, this plot means that people in their 30s and 40s are the most among all subscribers. We guess they had subscribed a deposit to prepare for their old age and a stable future. On the other hand, the elderly(60s~90s) have a higher 'yes_ratio' than 'no_ratio'. In other words, elderly have a higher ratio of subscribers than non-subscribers. It means that if the 'age' of the data is older, the probability of subscribing is relatively high.

1.3 job

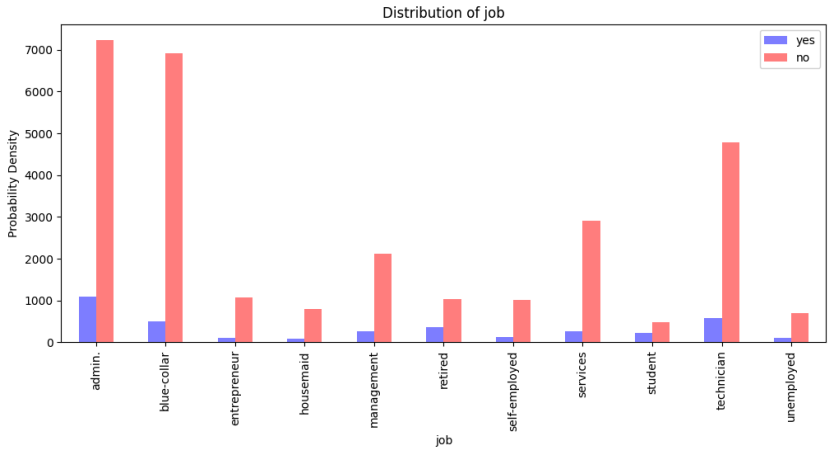


Figure 2: Absolute counts of 'job'

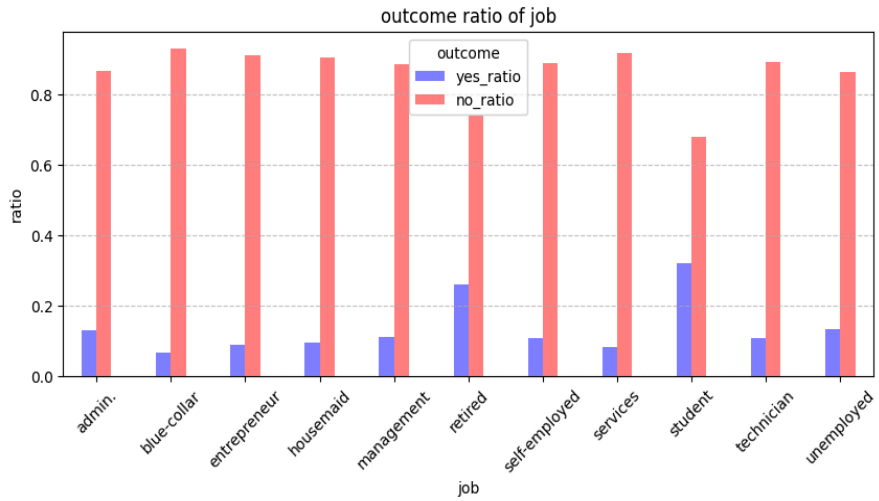


Figure 3: Ratio of 'yes' and 'no' of 'job'

Figure 2 shows absolute counts for each category of 'job'. The absolute counts of data are most distributed in 'admin.' and 'blue-collar'. In addition, we can identify that data with 'no' label (non-subscriber) is much larger than that with 'yes' label (subscriber) in all occupational groups. At this point, we thought that it would be important to identify proportions of 'yes' and 'no' for each category, by dividing 'yes' counts and 'no' counts by the total counts respectively. Thus, we also plot Figure 3. It shows the proportions of 'yes' and 'no' for each category of 'job'. Through this plot, we thought that 'retired' and 'student' categories will be important in class prediction since the 'yes_ratio' of these categories is higher. Also, we could get an insight that even though the absolute number of data was much higher in 'admin.' and 'blue-collar', the 'yes_ratio' and 'no_ratio' are pretty similar among categories. Therefore, we will plot for both absolute counts and ratio of 'yes' and 'no' for the rest of features as well.

1.4 marital

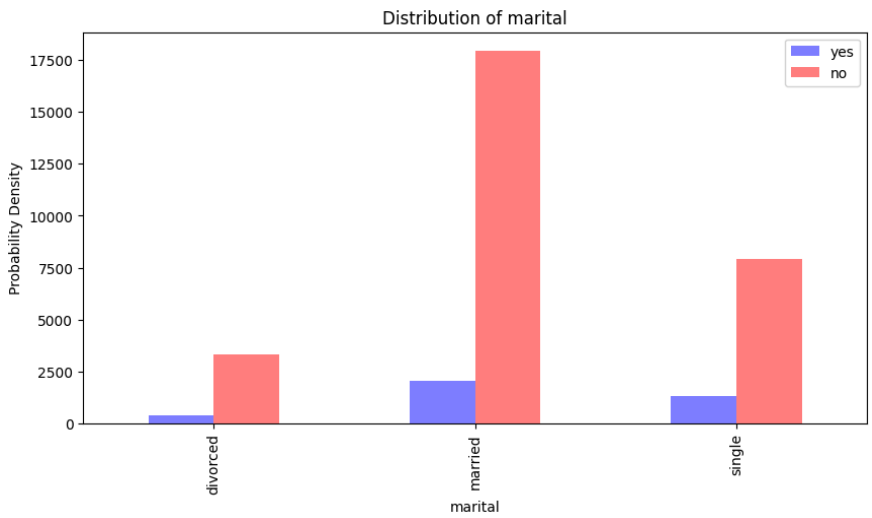


Figure 4: Absolute counts of 'marital'

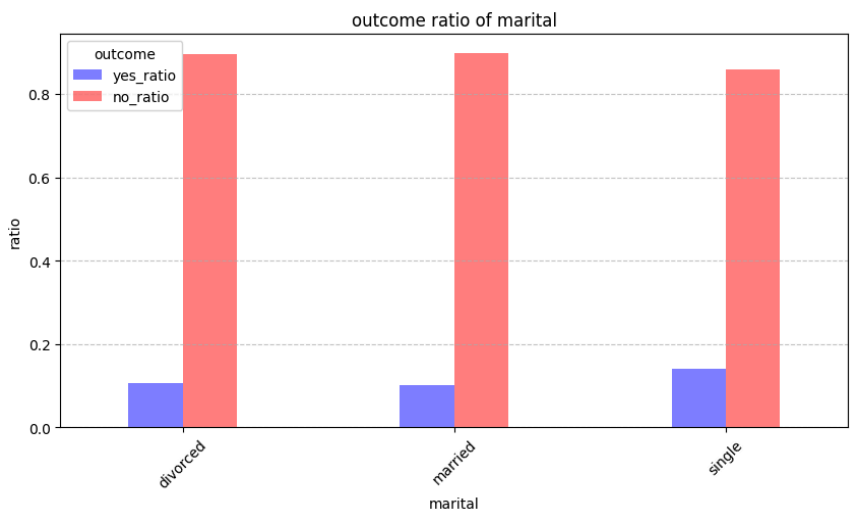


Figure 5: Ratio of 'yes' and 'no' of 'marital'

Figure 4 shows absolute counts of 'marital'. The absolute counts of data are distributed in 'married'. Figure 5 shows the proportions of 'yes' and 'no' for each category of 'marital'. By simply looking at Figure 4, there are many 'married' clients who subscribed. However, the proportion of 'yes' and 'no' for 'divorced', 'married', and 'single' are similar. As a result, it is a misconception that expecting the 'married' people will join the deposit more, simply because there are many 'married' customers.

1.5 Education

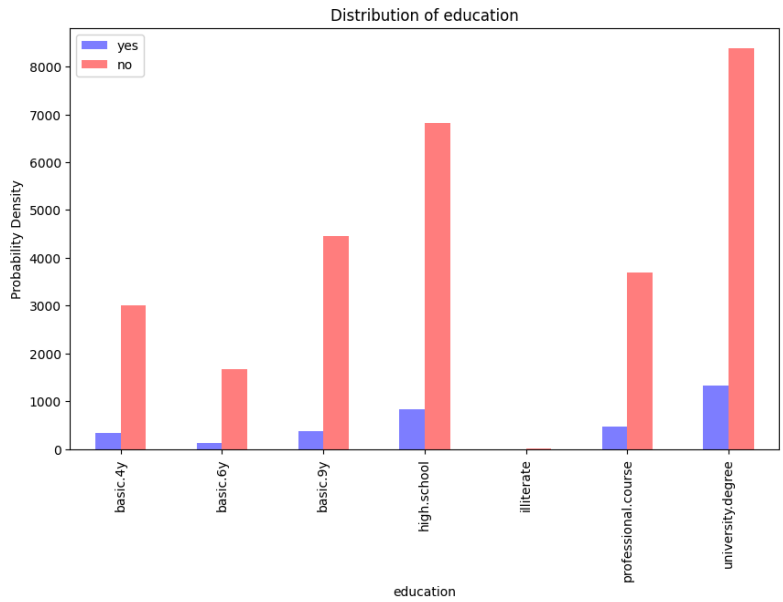


Figure 6: Absolute counts of 'education'

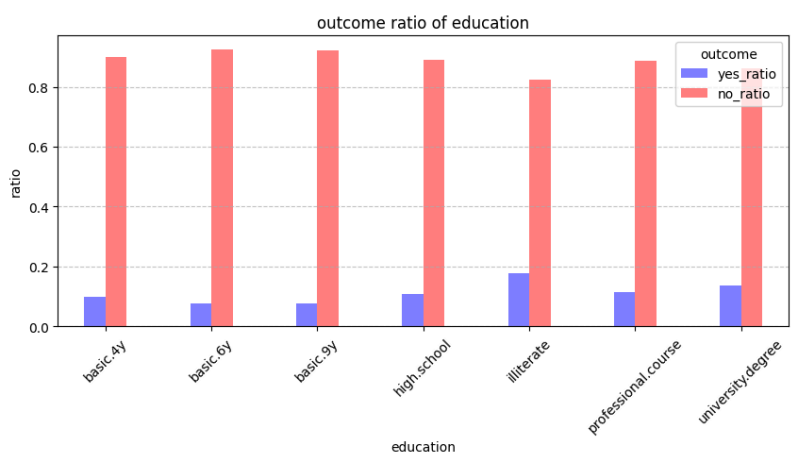


Figure 7: Ratio of 'yes' and 'no' of 'education'

Figure 6 shows absolute counts of 'education'. Through the visualization, we could identify that the absolute 'education' data of subscribers are more distributed in 'high.school' and 'university.degree'. Figure 7 shows the proportions of 'yes' and 'no' for each category of 'education'. It shows that 'illiterate' has a higher ratio of 'yes' than 'no' compared to other categories.

1.6 Default

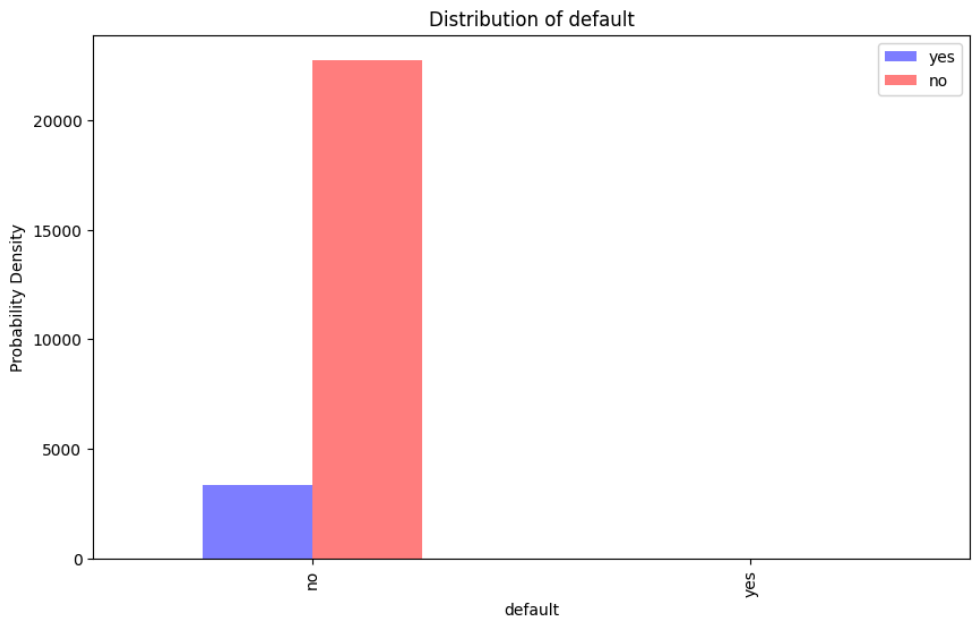


Figure 8: Absolute counts of 'default'

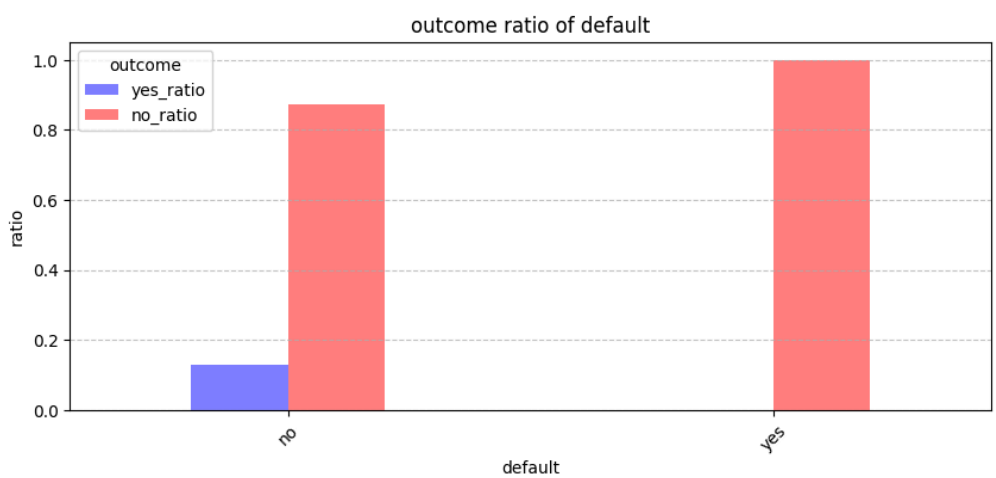


Figure 9: Ratio of 'yes' and 'no' of 'default'

Figure 8 shows absolute counts of 'default'. There are only three people who have a value of 'yes' in the total data. This is very small compared to the number of total data. Figure 9 shows the proportions of 'yes' and 'no' for each category of 'default'.

1.7 Housing

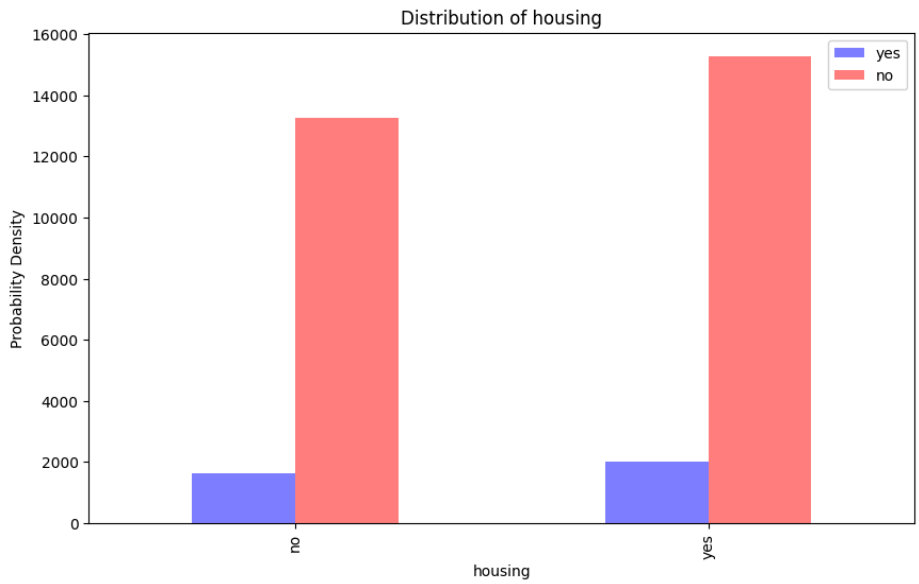


Figure 10: Absolute counts of 'housing'

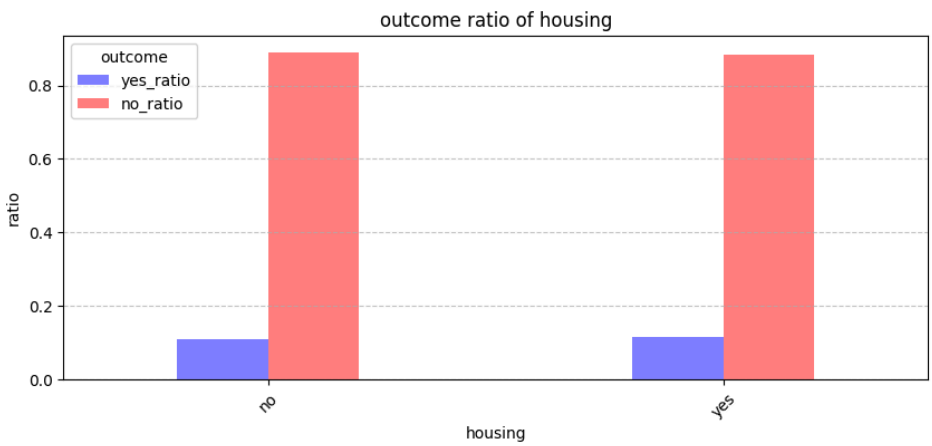


Figure 11: Ratio of 'yes' and 'no' of 'housing'

Figure 10 shows absolute counts of 'housing'. We could identify that 'yes' counts and 'no' counts are quite similar even though 'yes' counts are slightly higher, which means there is no notably dominant category. Figure 11 shows the proportions of 'yes' and 'no' for each category of 'housing'. And we can see that the 'yes_ratio' and 'no_ratio' of 'no' category and 'yes' category are very similar.

1.8 Loan

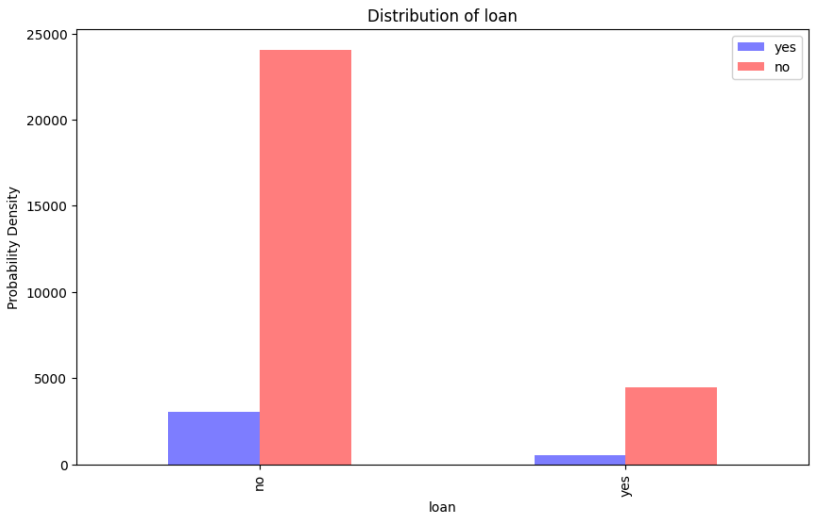


Figure 12: Absolute counts of 'loan'

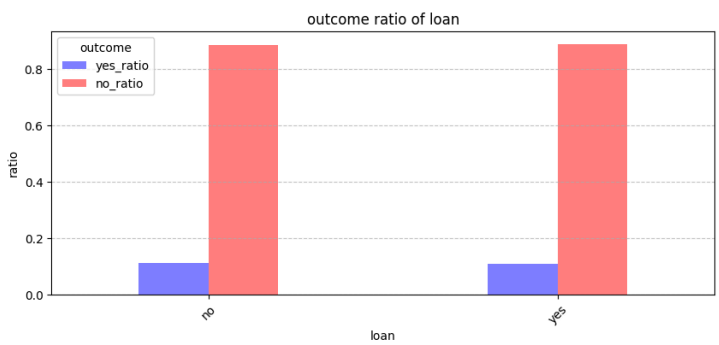


Figure 13: Ratio of 'yes' and 'no' of 'loan'

Figure 12 shows absolute counts of 'loan'. It shows that among all subscribers, the number of clients who do not have loans is much higher than that of clients who have loans. Figure 13 shows the proportions of 'yes' and 'no' for each category of 'loan'. We could see that the 'yes_ratio' and 'no_ratio' of 'no' value and 'yes' value are very similar.

1.9 contact

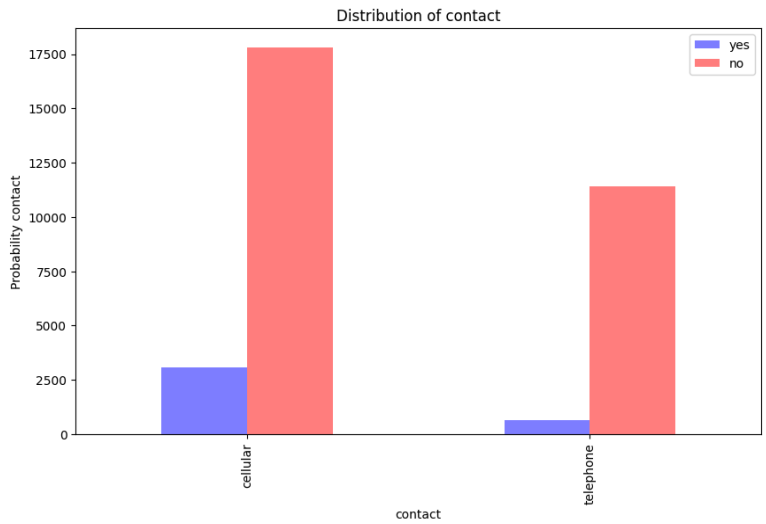


Figure 14: Absolute counts of 'contact'

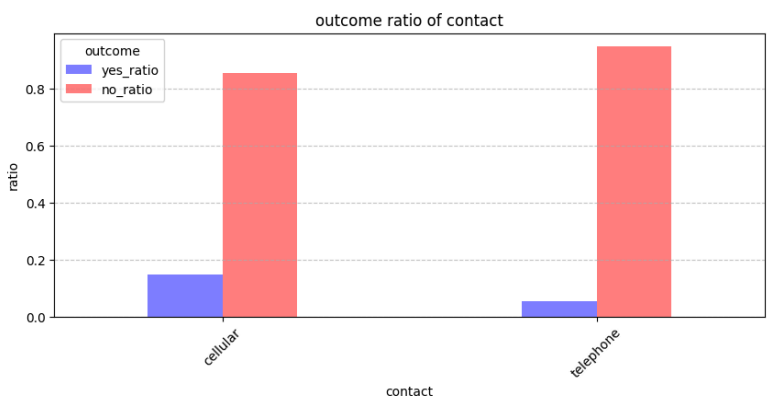


Figure 15: Ratio of 'yes' and 'no' of 'contact'

Figure 14 shows absolute counts of 'contact'. It shows that the number of clients who are contacted with 'cellular' are much higher than that of clients who are contacted with 'telephone'. Figure 15 shows the proportions of 'yes' and 'no' for each category of 'contact'. Looking at Figure 15, the 'yes_ratio' of when contact was attempted with 'cellular' is higher than that of when contact was attempted with 'telephone'. It means that contacting 'cellular' is more effective since most people are highly exposed to their own smartphones ('cellular') nowadays. Through this, we can know that both 'cellular' and 'telephone' are important in model prediction.

1.10 Month

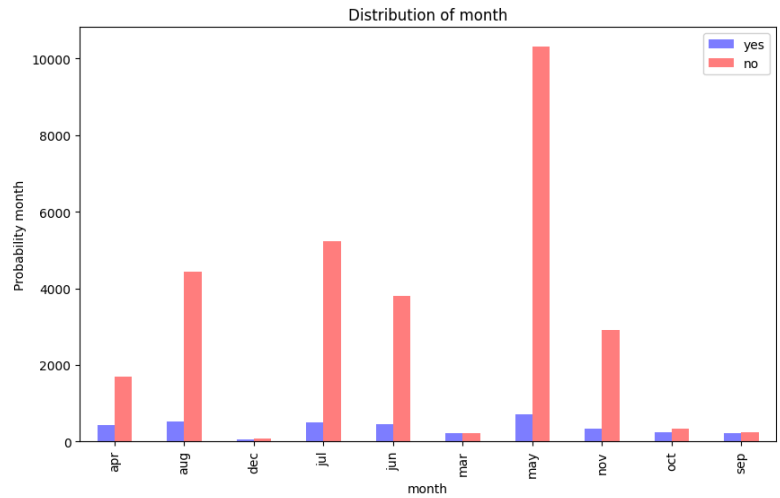


Figure 16: Absolute counts of 'month'

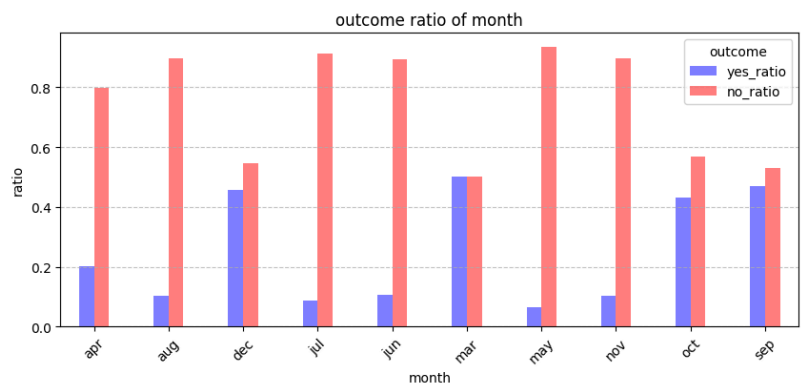


Figure 17: Ratio of 'yes' and 'no' of 'month'

Figure 16 shows absolute counts of 'month'. It shows that the number of clients who are contacted in 'may' was notably higher than that of clients who are contacted in the rest of the months. Also, there are only 'apr', 'aug', 'dec', 'jul', 'jun', 'mar', 'may', 'nov', 'oct', and 'sep'. Figure 17 shows the proportions of 'yes' and 'no' for each category of 'month'. Through this, we found that the 'yes_ratio' of 'dec', 'mar', 'oct', and 'sep' was notably higher than other months. Therefore, it can be seen that the subscription probability will be high for clients whose last contact month of year is December, March, October, and September.

1.11 day of week

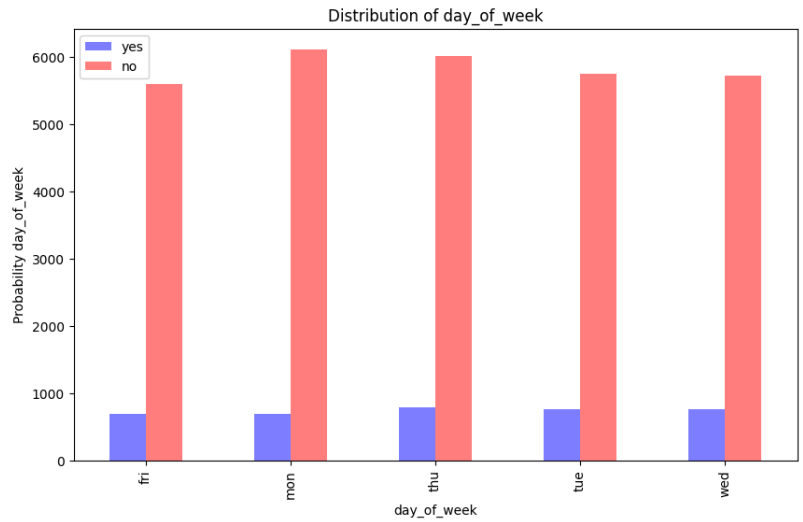


Figure 18: Absolute counts of 'day_of_week'

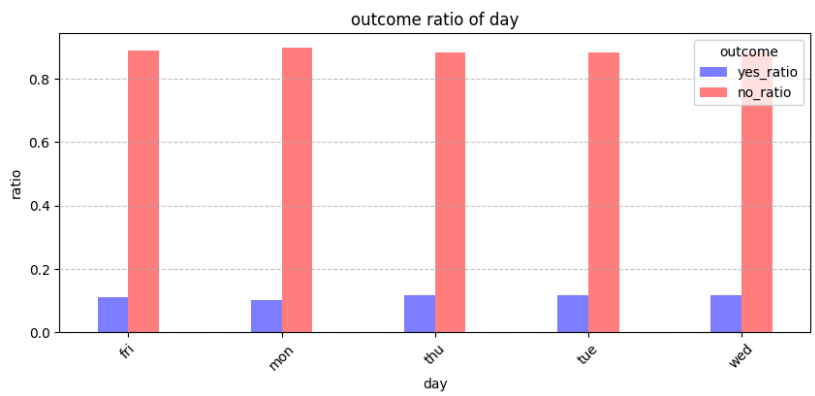


Figure 19: Ratio of 'yes' and 'no' of 'day_of_week'

Figure 18 shows absolute counts of 'day_of_week'. We could identify that counts of all days are quite similar. It means that there is no notably dominant category. Figure 19 shows the proportions of 'yes' and 'no' for each category of 'day_of_week'. We could see that the 'yes_ratio' and 'no_ratio' of every day are very similar in Figure 19.

1.12 campaign

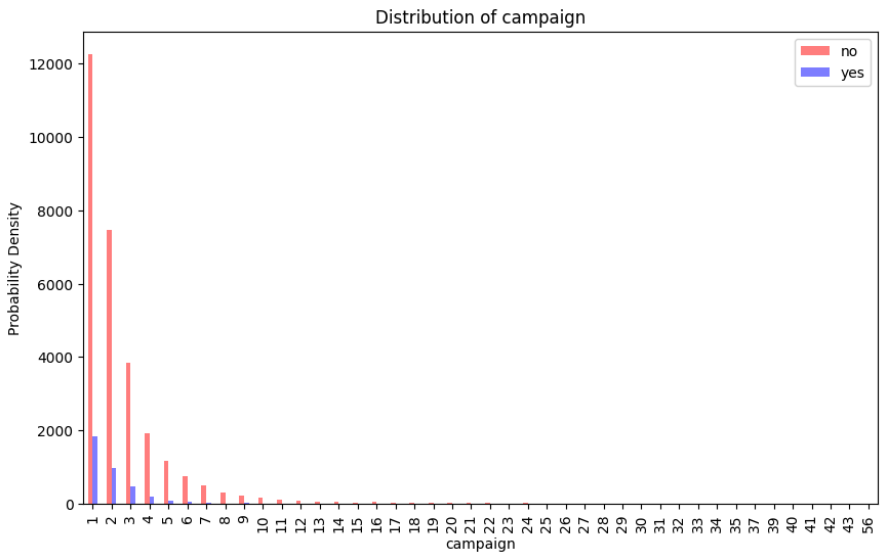


Figure 20: Absolute counts of 'campaign'

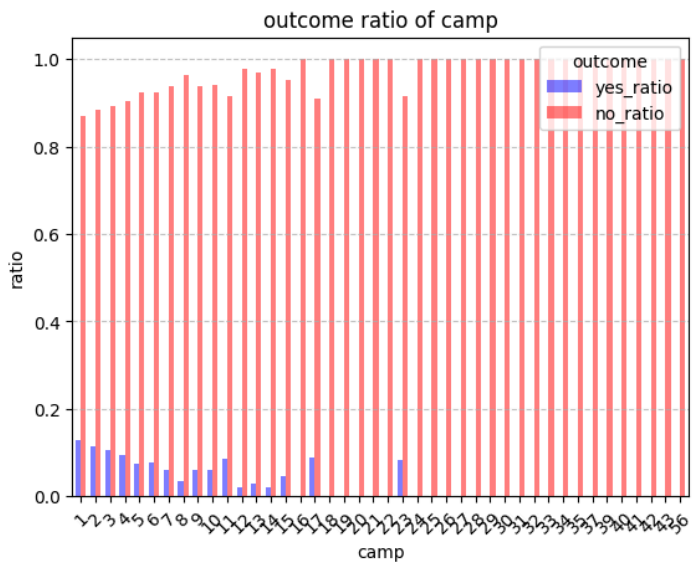


Figure 21: Ratio of 'yes' and 'no' of 'campaign'

Figure 20 shows absolute counts of 'campaign'. We could identify that counts of '1' and '2' are quietly dominant compared to others. Figure 21 shows the proportions of 'yes' and 'no' for each category of 'campaign'. We could see that most of the 'yes_ratio' occurred at category ''1' to '11', '17', and '23'.

1.13 Pdays

This variable is the number of days that passed by after the client was last contacted from a previous campaign. If it has a '999' value, it means the client was not previously contacted. So, we counted the number of '999' in the total train dataset and we found that most of the total data have '999' value. So, this feature is regarded as meaningless and we can drop this in the data preprocessing process.

1.14 Previous

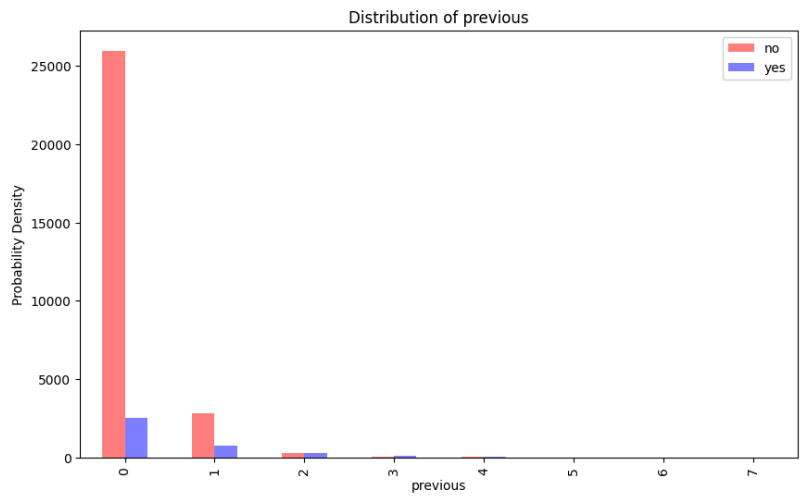


Figure 22: Absolute counts of 'previous'

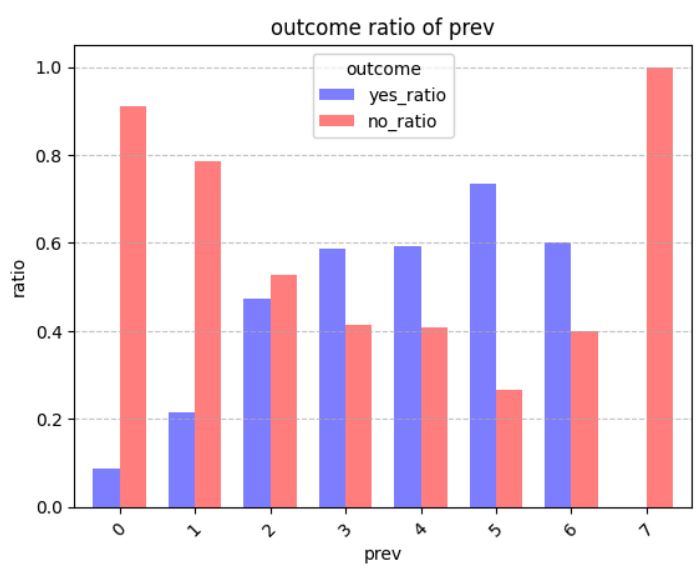


Figure 23: Ratio of 'yes' and 'no' of 'previous'

As a result of visualizing the ‘previous’ variables of subscribers, most cases are never contacted before starting the campaign in Figure 22. Additionally, like Figure 23, when visualizing the ‘yes_ratio’ and ‘no_ratio’ of the ‘previous' feature relatively, the ‘yes_ratio’ was high in the case that the 'previous' value equals 3, 4, 5 and 6. This means that the joining of bank deposits is relatively high, when the number of contacts before implementation of the campaign is equal to 3 to 6.

1.15 Poutcome

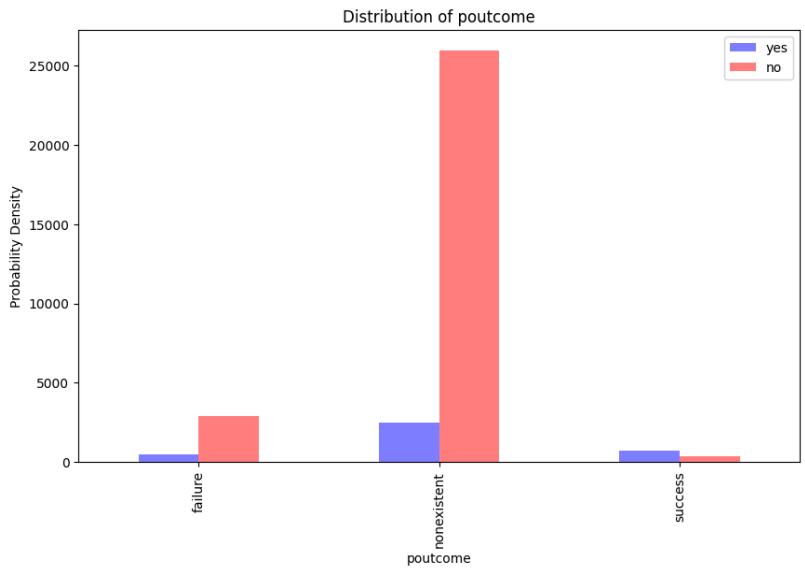


Figure 24: Absolute counts of ‘poutcome’

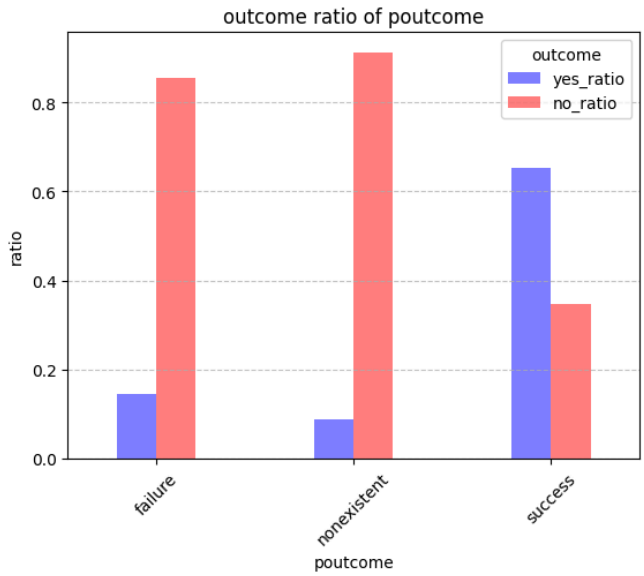


Figure 25: Ratio of 'yes' and 'no' of 'poutcome'

Figure 24 shows absolute counts of 'poutcome'. Through this visualization, we could identify that the 'nonexistent' category takes account dominantly for total data. It seems that there is no significant difference in the number of subscribers for each 'failure' value and 'success' value. However, Figure 25 shows a surprising result. Figure 25 shows the ratio of 'yes' and 'no' for each category of 'poutcome'. We could identify that the probability of subscribing to a deposit is much higher in the 'success' category than 'failure' and 'nonexistent' categories. Even though the 'nonexistent' category has the highest number of data, the actual ratio of 'yes_ratio' was the highest in the 'success' category. Through this, we identified that examining the proportion of 'yes' and 'no' of 'y' is a very important process in EDA.

1.16 emp.var.rate

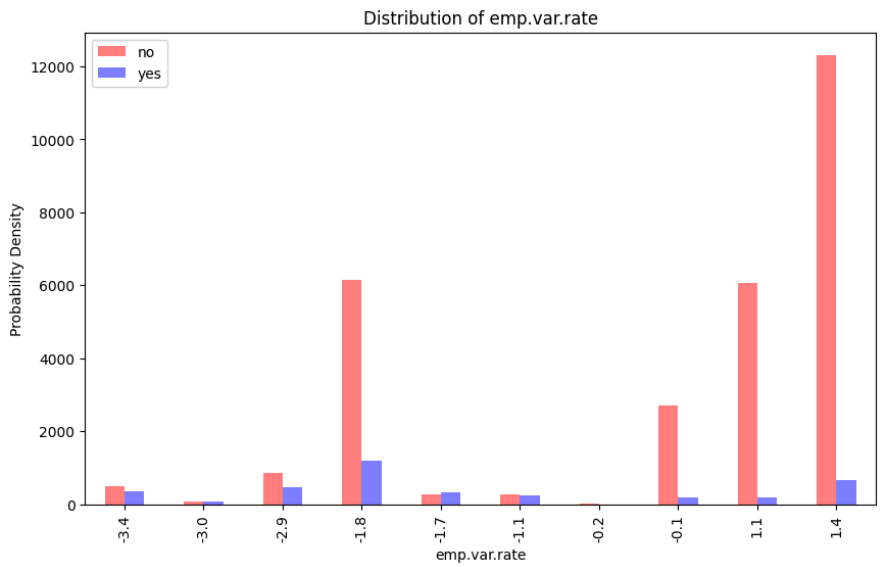


Figure 26: Absolute counts of 'emp.var.rate'

Figure 27: Ratio of 'yes' and 'no' of 'emp.var.rate'

'emp.var.rate' is a variable representing employment variation rate, and if this value increases in a positive direction, the number of employed people has increased compared to the previous point, and if the value increases in a negative direction, the number of unemployed people has increased. When looking at Figure 26, the number of subscribers is

higher in about -2.9 to -1.1 compared to the case when the value of 'emp.var.rate' is more large. It can be understood as a tendency for people to save money is higher when 'emp.var.rate' is moderately low rather than when it is large because they think they may lose their jobs.

1.17 cons.price.idx

'cons.price.idx' is a variable representing the consumer price index by month, which means the price that consumers feel. We guessed that if this value is higher, the consumer perceives the price is higher and their purchasing power will decrease, so the ratio of subscribing to deposit will increase.

1.18 cons.conf.idx

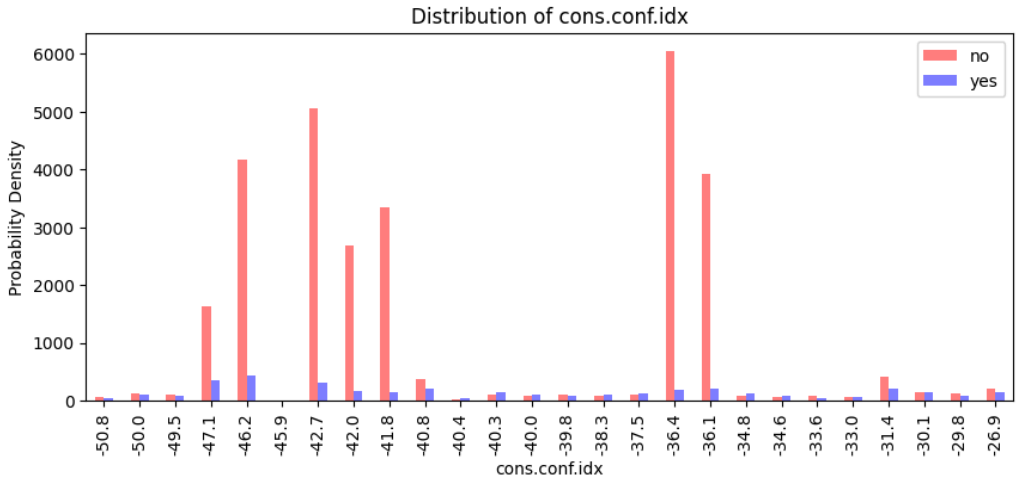


Figure 28: Absolute counts of 'cons.conf.idx'

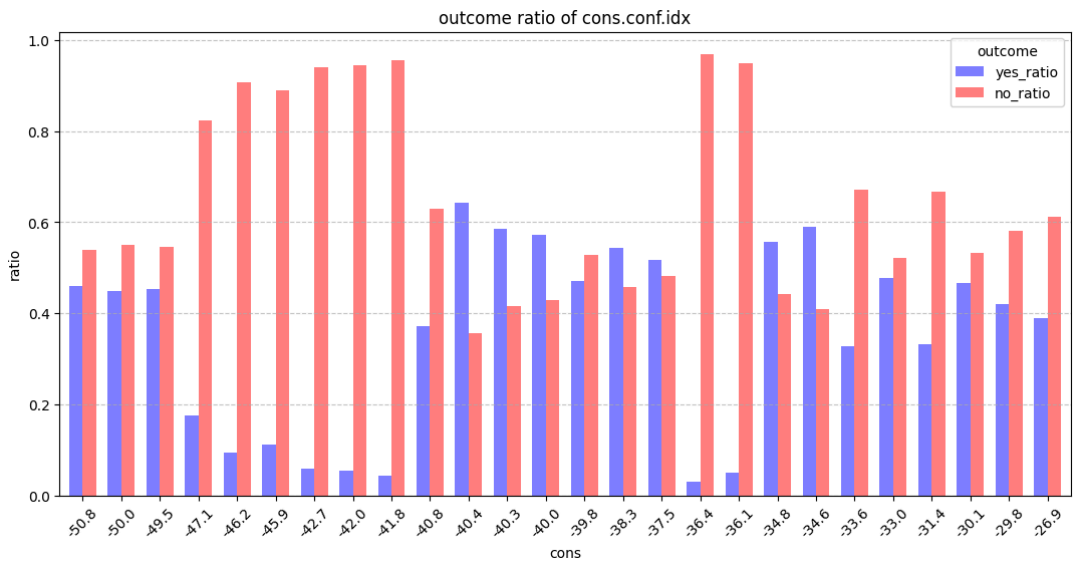


Figure 29: Ratio of ‘yes’ and ‘no’ of ‘cons.conf.idx’

‘cons.conf.idx’ is a variable representing the consumer confidence index, which quantifies the degree of confidence consumers feel about the economic situation. The lower the value of the variable, the more pessimistic consumers view the country's economic situation, and in this situation, consumers tend to increase savings and reduce consumption. Therefore, we assumed that the lower the value of the variable, the more people would subscribe to a deposit to prepare for the future. However, as a result of visualization of the train data, it was found that the small value of the variable did not necessarily increase the number of people who subscribe to a deposit. In Figure 28, among all subscribers, this value is most often around -46. And in Figure 29, when the value of ‘cons.conf.idx’ is -40.4 ~ -37.5, the probability of subscribing to a deposit (‘yes_ratio’) was relatively higher than the probability of not subscribing(‘no_ratio’).

1.19 euribor3m

Each country in Europe lends and borrows current assets from each other by measuring interest rates based on the Euribor rate. In other words, this variable refers to the market interest rate.

1.20 nr.employed

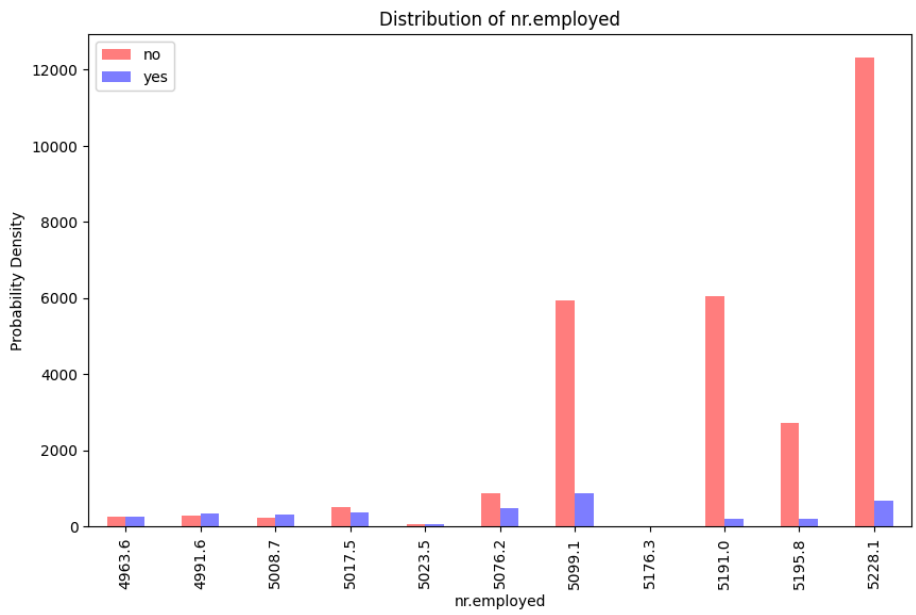


Figure 30: Absolute counts of ‘nr.employed’

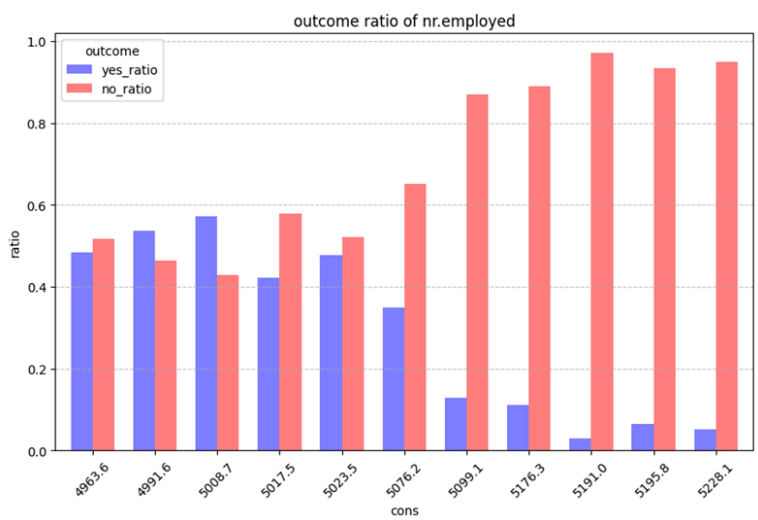


Figure 31: Ratio of ‘yes’ and ‘no’ of ‘nr.employed’

As a result of visualizing the 'nr.employed' of the subscribers in Figure 30, the absolute number of data is most distributed at values ranging from about 5,050 to 5,100. Additionally, looking at the relative ratio in figure 31, ‘yes_ratio’ is higher than ‘no_ratio’, when it has a value of about 4,980 to 5,000.

2. Heatmap

In order to find out the correlation between variables, we drew a heatmap for all numerical variables. Since it is impossible to analyze the relationship of all variables one by one, we found out the relationship of several variables at once through this heatmap.

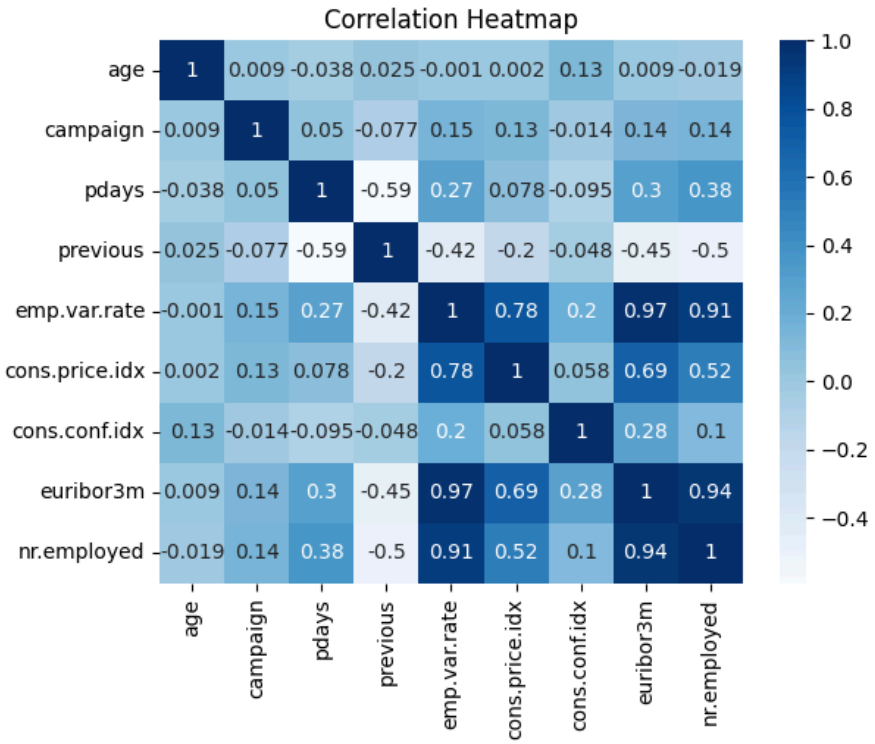


Figure 32: correlation heatmap for all numerical variables

According to this heatmap, there is a high correlation between 'euribor3m' and 'emp.var.rate' (0.97), 'euribor3m' and 'nr.employed' (0.94), and 'nr.employed' and 'emp.var.rate' (0.91). Additionally we investigated the relationship between 'euribor3m' and 'emp.var.rate'. At first, we took an economic approach and predicted that 'emp.var.rate' would decrease as the base interest rate('euribo3m') increases.

However, in Figure33, as a result of setting the x-axis to 'euribor3m' and the y-axis to 'emp.var.rate', the higher the value of 'euribor3m', the higher 'emp.var.rate', and the lower 'euribor3m', the lower 'emp.var.rate'. The results were different from our expectations, so we assumed that the social and economic background factors had an influence.

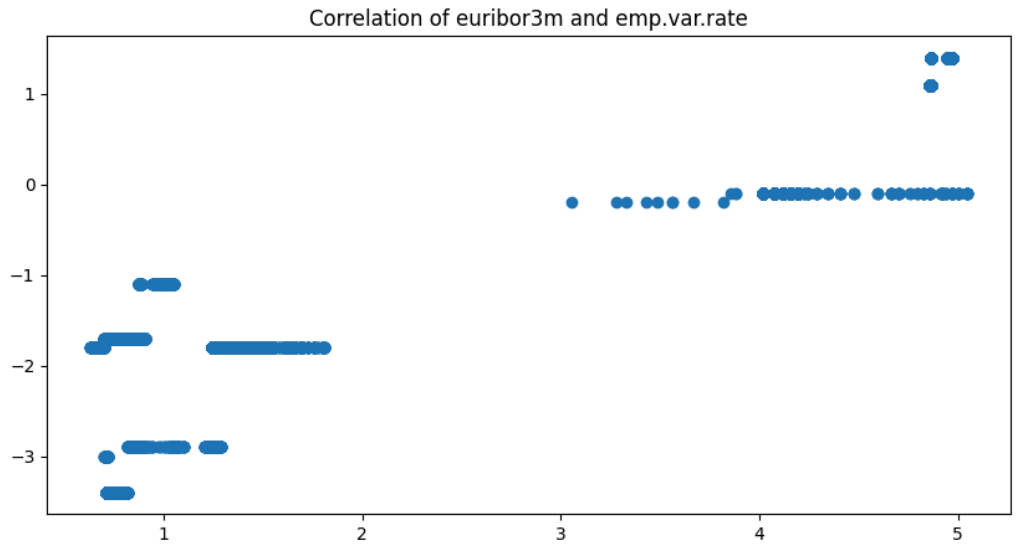


Figure 33: correlation scatter plot between 'euribor3m' and 'emp.var.rate'

In Figure 32, this heatmap can only analyze the correlation among numerical variables, so we built other specific hypotheses of the correlation of categorical values separately.

3. Hypothesis

3.1 [Hypothesis 1] 'The housing loan and loan will be proportional.'

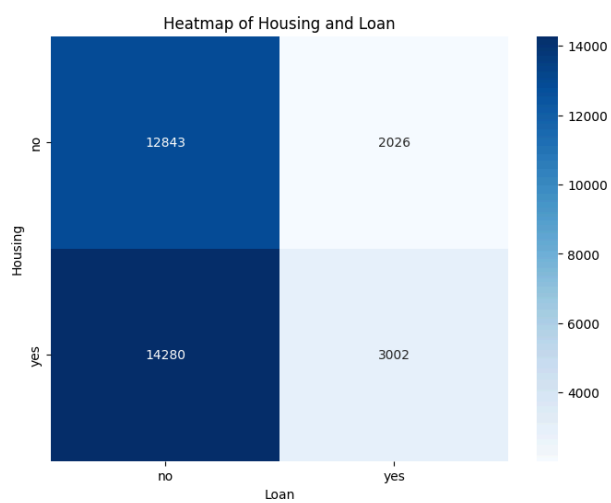


Figure 34: Heatmap of 'housing' and 'loan'

We decided to focus on the attribute 'loan' and surveyed the relationship between the 'housing' and 'loan' features. The variable 'housing' and 'loan' both use the concept of 'loan' idea, so we estimated that those two attributes would have similar inclination. The result of plotting the heatmap of the relationship between 'housing' and 'loan', the case that has 'loan' but no 'housing' is the most frequent. After thinking about what it suggests, the presence of housing loans means that the customer has their own house, so rather they are financially stable. However, the presence of a loan means an economically ill-considered situation. Consequently, hypothesis 1 is incorrect and we must consider 'housing' and 'loan' attributes as independent variables.

3.2 [Hypothesis 2] 'There should be a difference in the degree of acceptance of contact depending on the month.'

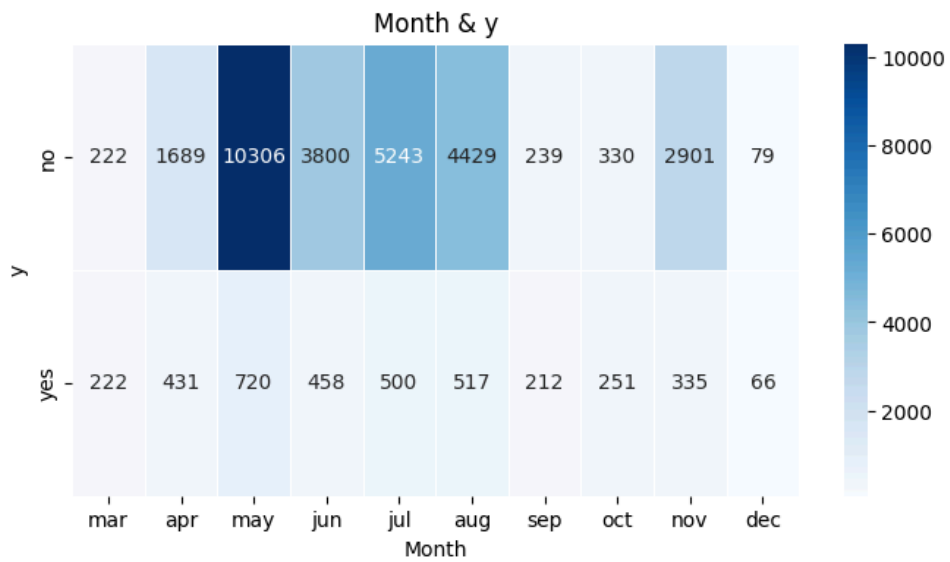


Figure 35: Heatmap of 'month' and 'y'

As a result of checking the proportion of success and failure of contact based on each 'month' attribute, the 'month' that has the highest success percentile is March and December. We estimate that the reason for high deposit success is because of a desire to start anew. For this reason, we can check hypothesis 3 is correct.

II. Data Pre-processing

1. Data Cleaning

1.1 Discovering the missing value

First, we replaced the value 'unknown' into NaN(np.nan), and printed every Null value in each column. There were 257 missing values in the 'job', 62 in the 'marital', 1388 in the 'education', 6853 in the 'default', and 799 in each 'housing' and 'loan', as Figure 34 shows.

```
id                  0
age                 0
job               257
marital            62
education        1388
default          6853
housing           799
loan              799
contact             0
month               0
day_of_week         0
campaign            0
pdays               0
previous            0
poutcome            0
emp.var.rate        0
cons.price.idx      0
cons.conf.idx       0
euribor3m           0
nr.employed         0
y                   0
dtype: int64
```

Figure 36: the number of NaN value of all attributes

1.2 Detecting outliers

Before filling missing values in every column, we found the outlier of columns to remove or replace them. This is because outliers, which are extremely small or big values in the outside of the range of total values, make the quality of the data be lower and further they lead to a bad performance of the model. Figure 35 shows several box plots based on numerical attributes, and we drew out that there are outliers only in the attribute 'age', 'campaign', 'previous', 'pdays', and 'cons.conf.idx'. However, we realized that not all outliers have to be removed since sometimes data can be treated as outliers wrongly.

Let us explain additionally regarding this insight. For the 'age' attribute, although the outliers are extreme, they may reflect the realism of the data. This means that the oldest and youngest customer in the data could be important. Additionally, to prevent the

distortion of interpretation, we decided to keep the ‘age’ outliers. Similar to this reason, ‘campaign’ has a very large range of values, so the big number of outliers just represents the diverse values of columns. Also, ‘previous’ has discrete values, and according to that characteristic, the outlier of the ‘previous’ column can be ignored. Additionally, ‘pdays’ is configured majorly as ‘999’ value (refer to Figure 36), so non-999 values can be thought of as outliers. However, they are not the actual outliers, so we got an insight that we should also consider carefully what are the true outliers that must be actually removed.
For these reasons, we did not drop or fill outliers in other values. In contrast, there are some outliers in the ‘cons.conf.idx’ column. Therefore, we made a decision to remove outliers of ‘cons.conf.idx’ and fill them with the max value of ‘cons.conf.idx’.

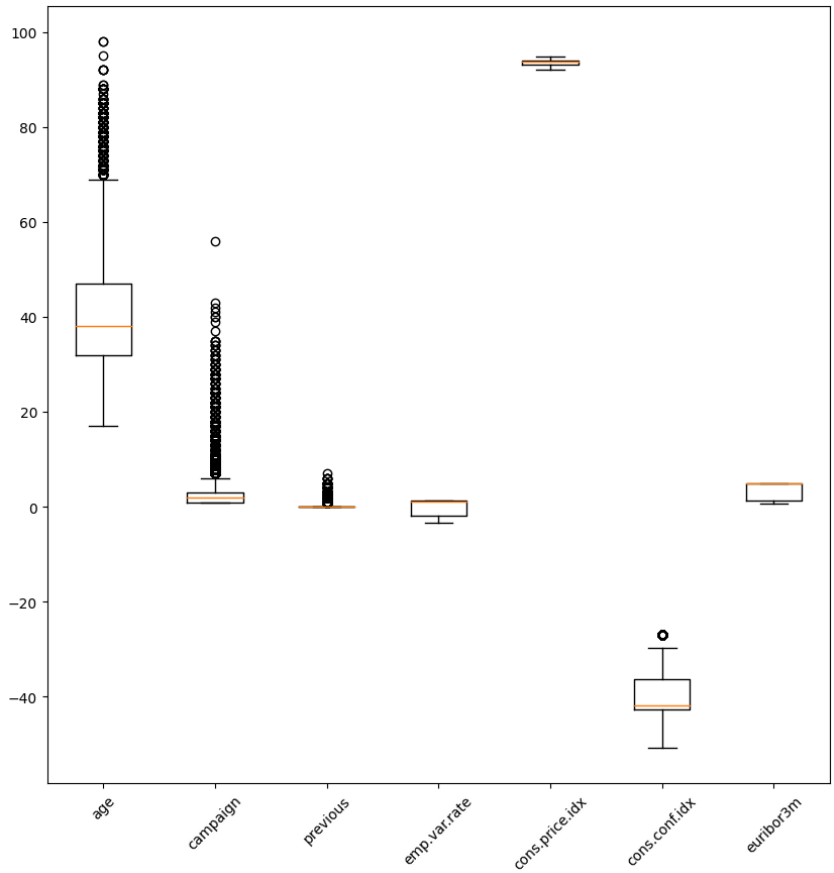
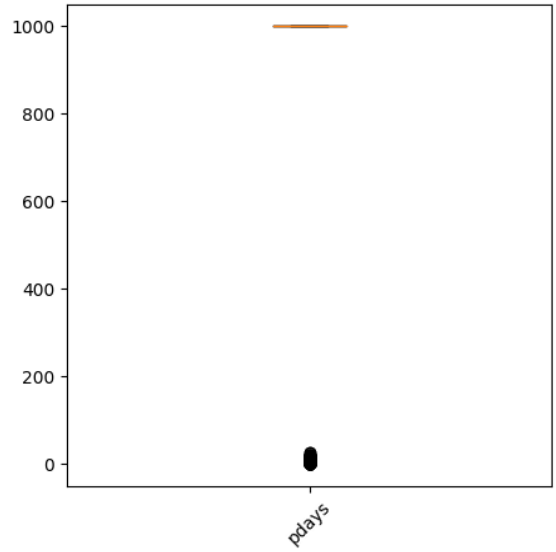


Figure 37: the box plot for numerical attribute

Figure 38: the box plot of the attribute ‘pdays’



The code which removes outliers based on IQR is shown in Code 1. It defines the lower bound, the spot of the 25th percentile(first quartile), and the upper bound, the spot of the 75th percentile(third quartile). Subsequently, it calculates the IQR value, which is the subtraction of upper bound and lower bound. The standard of dropping the outlier is addition and subtraction of the 1.5 multiple of IQR, so it leaves the data in scope. Continually, the code calculates the max value among the inner-range of the box plot, and replaces outliers with it. Actually, the ordinary method to fill the numeric data is using the mean value of themselves. However, Figure 35 shows that outliers of the attribute 'cons.conf.idx' are only on the bigger side of the box plot. At this point, we thought that we could advance the existing concept of filling the missing value of numerical variables. Combine the Winsorizing, which replaces the max value among normal ones, we modify the code(refer to Code 1).

```
[ ] def remove_outliers_box(data, columns):

        data_df = data.copy()

        for col in columns:

            Q1 = data_df[col].quantile(0.25)
            Q3 = data_df[col].quantile(0.75)
            IQR = Q3 - Q1

            lower_bound = Q1 - 1.5 * IQR
            upper_bound = Q3 + 1.5 * IQR

            max_value = data_df.loc[(data_df[col] >= lower_bound) & (data_df[col] <= upper_bound), col].max()
            data_df.loc[(data_df[col] <= lower_bound) | (data_df[col] >= upper_bound), col] = max_value

        return data_df
```

Code 1: the code of removing the box plot outliers

1.3. Replacing missing values

1.3.1 Based on the proportion of attributes

According to the 1.1. and 1.2., there are missing values in 'job', 'marital', 'education', 'default', 'housing', 'loan', and 'cons.conf.idx'. We decided to fill the 'job', 'marital', 'default', 'loan', based on the characteristics of attributes. For example, as the bar chart of relative 'job' distribution(refer to Figure 0) shows, the relative job distribution is not evenly distributed but severely biased towards 'admin.' and 'blue-collar'. Therefore, we thought using the mode value to fill in all NaN values can cause critical error. For this reason, we decided to replace the missing value, employing the proportion of existing data's distribution. Let me take a 'job' as an example. First, calculate the proportion of each category in 'job' and add it to 'job_dict' which has each category name as keys and its proportion as values. Next, using these proportions, calculate counts for each category to fill in NaN values of 'job'. Then, extract a data frame named 'trained_job_null' that has 'job' value as NaN. Replace missing values with designated counts of each category. Finally, drop rows with missing values of 'job' from train(original data frame) and concatenate that to 'trained_job_null'. Apply the same processes to 'marital', 'education', 'default', 'housing', and 'loan' as well.

1.3.2 Based on KNN Imputer

Now, there are missing values only in 'education' and 'housing' attributes. Unlike other features, the relative distributions of left attributes are almost even. Thus, we thought that

we can apply the concept of KNN(K-nearest neighbor) to impute categorical values. Normally, the KNN is used to fill the missing value of numerical values, but we advanced it to handle the categorical attributes. As a result, to utilize the KNN imputer, data must have a type of integer or float. Thus, we encoded the categorical values in numbers, using the label encoder method.

While using label encoding, we have to be aware of a side effect that label encoder converts NaN values as one independent label. Therefore, we have to change the value '7' in the 'education' and the value '2' in the 'housing' variable as NaN(np.nan). Then we imported the 'KNNImputer' in the 'scikit-learn' module, and performed the replacement. As for dealing the categorical variables as numerical, the imputed results are float. Thus, we converted that float values into a type of integer, simply by rounding the float values. Finally, we converted values of categorical features which now have int types into string type in order to get dummy variables of those in later.

2. Data Reduction

2.1 Dropping and Encoding the data

We want to drop several columns which are less important to predict the bank term deposit to reduce computational efficiency and prevent overfitting problems. Therefore, we dropped obviously irrelevant features, like 'id', 'pdays', and 'y'. Then we tried to encode the rest categorical features by getting the dummy variables of those. Now, we normalized the dataset employing zero-centering using Code 2. Lastly, we split the dataset into train and test datasets. As a result, we use these datasets as the input of the selection method based on stepwise feature selection.

```
[ ] # zero centering
    data-=data.mean(axis=0)
    data
```

Code 2: the code of zero-centering

2.2 Feature Selection using the Stepwise Method

We used a stepwise method since the number of dimensions is 57, caused by getting dummies of categorical features. It can occur overfitting and degrade the performance of a model. Also, the interpretation of a model will be improved by selecting highly relevant features to the target ('y').

Wrapper method is one of the types of feature selection, which uses a predictive model to find the optimal subset of features. Stepwise method is one of the techniques of the wrapper method. It is a combination of forward selection and backward elimination, adding or removing one feature recursively per each step. It considers all the cases of possible subsets of features and is able to find the best subset of features based on p-values, AIC, and BIC. Thus, it outperforms the forward selection or backward elimination in general.

To apply the process of the stepwise method, there are some statistical concepts that we have to understand. P-value represents the probability that the coefficient is actually zero. It means that the lower p-value is, the more significant the feature is. Generally, a feature is considered as statistically meaningful if the p-value of a feature is lower than 0.05. For example, feature 'age' is excluded in the optimal subset of features since it has higher p-value (0.866) than 0.05. AIC(Akaike Information Criterion) and BIC(Bayesian Information Criterion) are measures of the quality of a model for a given data. AIC is a statistic which considers both a model's fitness and complexity. The equation of AIC is follows:

$$AIC = 2K - 2ln(L)$$

L is the likelihood of the model's correct prediction, which needs to be maximized. K is the number of features, which has an effect of giving a penalty to a model which uses many parameters. Since our goal is to select the minimum number of features while maximizing the likelihood of the model's correct prediction, the model is considered better if it has lower AIC. BIC is similar to AIC but it gives larger penalties than AIC. However, BIC can moderate the overfitting problem occurring due to enlarging L. Following is the equation of BIC:

$$BIC = ln(n)k - 2ln(L),\ where\ n\ in\ the\ number\ of\ data\ points.$$

Same as AIC, the model is considered better if it has lower BIC. Adjusted R-squared is a solution of R-squared being too sensitive to the number of input features. It gives a penalty

when unnecessary features are added. The model is considered better if it has higher adjusted R-squared. Among these statistics concepts, we used p-values to determine whether to add or remove some features and Adjusted R-squared to identify the performance of a model.

We used the OLS(Ordinary Least Squares) regression model for our feature selection. It provides a systematic approach to fitting a model that best represents the relationship between independent variables and dependent variables, minimizing the sum of the squared residuals. Fits 'X_train' (independent variable) and 'y_train' (dependent variable) to a model, computing the coefficient of each feature of 'X_train' when predicting 'y_train'. Statsmodels package calculates p-values, AIC, and BIC and we could identify each feature's statistics by printing 'model.summary()'. With this model, we defined a stepwise method function.

In each step, compute p-values of features which are not selected and allocate the lowest p-value to 'min_pval'. If 'min_pval' is lower than 'sl_enter'(=0.05), append that feature to a list named 'selected_variables'. Then, select a feature which has the highest p-value among the selected variables and allocate the highest p-value to 'max_pval'. If 'max_pval' is higher than 'sl_remove'(=0.05), remove that feature from 'selected_variables'. Finally, compute Adjusted R-squared using 'selected_variables'. Repeat this step until there is no left feature anymore. We draw a plot which has the number of steps and list of selected features on x-axis and Adjusted R-squared on y-axis. Through this, we could visualize the process of the stepwise method.

As a result of feature selection, important attributes that the code select are 'default_0'('no'), 'default_1'('yes'), 'poutcome_2'('success'), 'month_6'('may'), 'month_5'('mar'), 'cons.conf.idx', 'poutcome_1'('nonexistent'), 'poutcome_0'('failure'), 'day_of_week_1'('mon'), 'month_4'('jun'), 'contact_0'('celluar'), 'contact_1'('telephone'), 'emp.var.rate', 'cons.price.idx', 'month_1'('aug'), 'euribor3m', 'month_7'('nov'), 'month_0'('apr'), 'job_1'('blue-collar'), 'day_of_week_4'('wed'), 'job_5'('retired'), 'campaign', 'job_8'('oct'), 'month_2'('dec'), 'month_9'('sep'), 'job_7'('services'), 'job_3'('housemaid'). Finally, we made our final dataset (X_train, X_test) which has selected features and values of those.

III. Model Training and Validation

1. The Problem of Imbalanced Dataset

```
[4] train['y'].value_counts()

y
no     29238
yes     3712
Name: count, dtype: int64
```

Code 3: Calculating the ratio of 'yes' and 'no'

As shown in Code 3, a prominent characteristic of our train dataset is that it is class-imbalanced. The ratio between 'yes' and 'no' is about 1 : 8. With an imbalanced dataset, it is very difficult to obtain a high f1 score. The reason why imbalanced dataset cannot obtain a high f1 score is as follows:

$$f1 = 2 * (precision * recall)/(precision + recall)$$

$$precision = TP/(FP + TP)$$

$$recall = TP/(TP + FN)$$

Precision and recall have the same numerator, but in the case of imbalanced data, the denominator of one of these metrics becomes significantly larger. And since the F1 score is the harmonic mean of precision and recall, it can only be high if both values are balanced. This is why we cannot obtain a high f1 score with a class-imbalanced dataset.

The objective of our project is to maximize the f1 score. So solving the class-imbalance problem is very crucial. Thus, we decided to focus on addressing this problem throughout the modeling process.

There are two ways of addressing this problem. The first approach is to match the class proportions through sampling. The second approach is to assign weights to the minor class. We ultimately chose the second approach, because it returned a higher f1 score. But we would also like to introduce the first approach, because we encountered meaningful

trial and error while attempting it.

2. First Approach - Resampling

First, we attempted balancing class proportions, using various resampling methods. Refer to Table 1.

method	NearMiss	SMOTE	ADAYSN	SMOTE-Tomek
f1	0.4809	0.4938	0.4778	0.4879

Table 1: f1 score based on various resampling methods

Among NearMiss, SMOTE, ADASYN, SMOTE-Tomek, we obtained the highest f1 score when using SMOTE. So for the rest of the report, we will only mention SMOTE.

After balancing the data, another issue arises. Of course balancing the data will increase the f1 score. However, it is questionable whether this necessarily guarantees an improvement in the model's performance.

This concern leads to our idea that sampling and training *multiple* times will guarantee the improvement of the model's performance. The problem of sampling just once is that the sample might not well express the entire data, thus the model can be biased to just a part of the data. By sampling multiple times randomly, we can say that we explored the entire dataset.

2.1. Constructing the Model

Based on this idea, we constructed the architecture of our model. As in Figure 39, we resample the train data n times. Then, we train n models based on each resampled dataset. Finally, we aggregate the predictions from the n models to derive the final prediction.

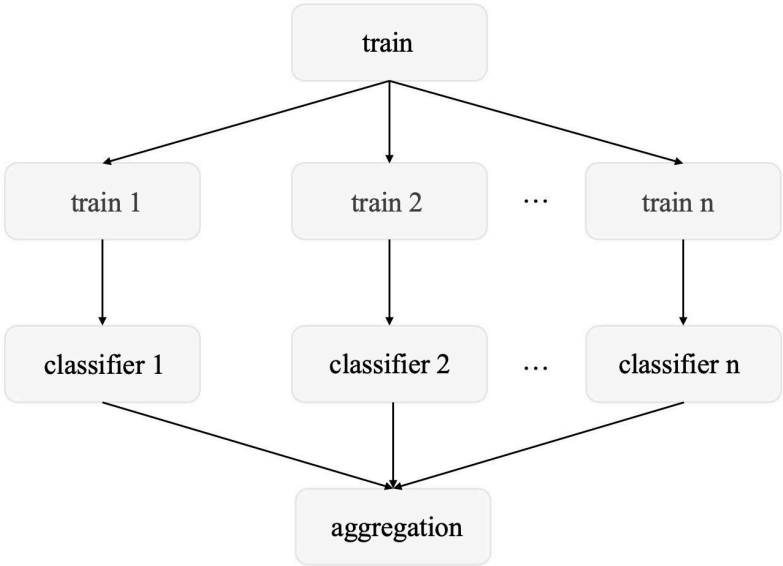


Figure 39: The architecture of our model

```
[205] def train (learning_rate,max_depth):

        # initialize number of models,
        # list for saving models and predictions,
        # and dataframe for saving P(y=1)

        n= 5
        GB_models=[]
        prob=pd.DataFrame()
        y_pred=[]

        # train model with resampled data
        for i in range(n):
          X_resampled,y_resampled=SMOTE(random_state=i).fit_resample(X_train,y_train) # resample
          model=GradientBoostingClassifier(random_state=i,learning_rate=learning_rate,max_depth=int(max_depth),n_estimators=100,n_iter_no_change=10)
          model.fit(X_resampled,y_resampled)
          GB_models.append(model) # save models in GB_models list

        # return P(y=1), predicted by models in GB_models list
        for trained_model in GB_models:
          y_prob=trained_model.predict_proba(X_test)[:,1]
          prob[f'{trained_model}']=y_prob

        tot_prob=prob.mean(axis=1) # caculate the avg of P(y=1) of all models in GB_models list

        # find the best threshold for maximum f1
        from sklearn.metrics import precision_recall_curve
        precision, recall, thresholds = precision_recall_curve(y_test, y_prob)
        f1_scores = 2*recall*precision/(recall+precision)
        threshold=thresholds[np.argmax(f1_scores)]

        # if the avg P(y=1) is bigger than threshold, predict as 1, else 0
        y_pred=np.where(tot_prob > threshold, 1, 0)

        # return f1 based on the prediction
        f1=f1_score(y_test, y_pred)
        return f1
```

Code 4: the codes of our first model

2.2. Code Implementation

Code 4 is the implementation of our first model.

First we initialize 'n', which indicates 'n' in Figure 39. Then we initialize the list 'GB_models', which we will save the multiple trained models in. In the 'prob' dataframe, we will save the predicted probability of 'y' being 1. Lastly, in list 'y_pred', we will save the predicted class for each datapoint.

Second, we resample and train models n times. In order to train each n model with n different sampled datasets, we set the random state as 'i' for both resampling and for the model. Then we append the trained model to list 'GB_models'. In Code 4 we used GradientBoostingClassifier. We selected GradientBoostingClassifier because it returned the highest f1 score among the models we tried out (refer to Table 2)

Model (Classifier)	Decision Tree	RandomForest	AdaBoost	GradientBoosting	XGB	LGBM
f1	0.4934	0.4935	0.4805	0.5	0.4932	0.4938

Table 2: f1 score based on classifier models

Third, we calculate P('y'=1) for all models in list 'GB_models', using the test set. Then we save those probability in one dataframe called 'prob'.

Next, we calculate the best threshold that maximizes the f1 score. Based on that threshold, we decide the class of the datapoint. That is, if P('y'=1) is higher than the threshold, we conclude that the class of that datapoint is 1.

Lastly, we returned the f1 score.

2.3. Bayesian Optimization

With Bayesian optimization, we found the best hyperparameter and f1 score for the model. (Refer to Table 3)

Note that these values can slightly change depending on the sampled data.

f1 score	learning rate	max_depth
0.4938	0.01412	7.358

Table 3: best f1 score and hyperparameters

3. Second Approach - Assign Weights

The next approach to handling imbalanced data is to assign higher weights to the samples of the minority class and lower weights to the majority class during the training process. By doing this, the model pays more attention to the minority class and learns to make better predictions for it.

To achieve this, we examined the APIs of the models tested in Table 2 to find those that support parameters related to class weights. As a result, I found that the LGBMClassifier can adjust the class weights for a binary classification task by setting 'is_unbalance=True.'

3.1. Code Implementation

Code 5 is the implementation of our second model. The overall structure of Code 5 is not very different from Code 4. The modifications made from Code 4 is as follows:

First, we eliminated the SMOTE resampling process. And we changed the model from GradientBoostingClassifier to LGBMClassifier, and changed parameters accordingly. Lastly, we added 'is_unbalance=True.'

```
[426] from lightgbm import LGBMClassifier
      def train (learning_rate,max_depth,n_estimators,num_leaves,reg_alpha,reg_lambda):

        # list for saving models and predictions,
        # and dataframe for saving P(y=1)
        LGBM_models=[]
        prob=pd.DataFrame()
        y_pred=[]

        # train model with parameter 'is_unbalance=True'
        model=LGBMClassifier(learning_rate=learning_rate,max_depth=int(max_depth),n_estimators=int(n_estimators),
                             num_leaves=int(num_leaves),reg_alpha=reg_alpha,reg_lambda=reg_lambda,
                             is_unbalance=True,verbose=-1)
        model.fit(X_train,y_train)
        LGBM_models.append(model) #save models in LGBM_models list

        # return P(y=1), predicted by models in LGBM_models list
        for trained_model in LGBM_models:
          y_prob=trained_model.predict_proba(X_test)[:,1]
          prob[f'{trained_model}']=y_prob

        tot_prob=prob.mean(axis=1) # caculate the avg of P(y=1) of all models in LGBM_models list

        # find the best threshold for maximum f1
        from sklearn.metrics import precision_recall_curve
        precision, recall, thresholds = precision_recall_curve(y_test, y_prob)
        f1_scores = 2*recall*precision/(recall+precision)
        threshold=thresholds[np.argmax(f1_scores)]

        # if the avg P(y=1) is bigger than threshold, predict as 1, else 0
        y_pred=np.where(tot_prob > threshold, 1, 0)

        # return f1 based on the prediction
        f1=f1_score(y_test, y_pred)
        return f1
```

Code 5: the codes of our second model

3.2. Bayesian Optimization

With Bayesian optimization, we found the most outperformed hyperparameter and f1 score for the model. (Refer to Table 4)

f1	learning_rate	max_depth	n_estimators	num_leaves	reg_alpha	reg_lambda
0.5076	0.1	135.4	223.0	10.0	0.0	0.1

Table 4: best f1 score and hyperparameters

VI. Interpretation and Analysis

Before we made an interpretation of the selected features and the result of model training, we extracted feature importances from the model which yields the highest f1 score. Figure 40 is the data frame of feature importance of selected features. Through this, we could identify that ‘euribor3m’ and ‘campaign’ were highly important to the model in predicting the target class. Also, ‘cons.conf.idx’, ‘cons.price.idx’, ‘day_of_week_1’(‘mon’), ‘day_of_week_4’(‘wed’), ‘emp.var.rate’, ‘contact_1’(‘telephone’) and ‘job_5’(‘retired’) showed notable importance compared to rest of the features. Based on Figure 40 and visualizations we made at first, we will interpret the meaning of selected features and how it affected the prediction of the model.

	feature	importance
15	euribor3m	722
21	campaign	376
7	cons.conf.idx	143
13	cons.price.idx	115
8	day_of_week_1	89
19	day_of_week_4	74
12	emp.var.rate	63
10	contact_1	62
20	job_5	61
6	poutcome_0	58
18	job_1	55
2	poutcome_2	49
25	job_7	42
5	poutcome_1	34
3	month_6	31
17	month_0	28
26	job_3	26
9	month_4	24
22	job_8	22
4	month_5	21
16	month_7	20
11	contact_0	15
14	month_1	12
24	month_9	12
23	month_2	6
1	default_1	0
0	default_0	0

Figure 40: Feature Importances

1. Interpretation of the selected features

1.1 job

Among 'job', 'blue-collar', 'housemaid', 'retired', 'services', and 'student' were selected. Looking at Figure 3, 'retired' and 'student' categories have high 'yes_ratio' compared with 'yes_ratio' of other categories. When related to the economic situation, since retired people do not have a fixed income, they want to receive a stable income. Students subscribed to a deposit to prepare for adulthood or for their specific goal. For this reason, we can guess that these two categories are selected. In the same context, 'blue-collar' and 'housemaid' are mainly engaged in physical labor, so they subscribe to deposits a lot because their income is not constant and unstable. Even in the case of 'services', they may not have a constant income since people employed in 'services' can always be replaced by others at any time. Therefore, these categories of 'job' are expected to play an important role in predicting whether clients will subscribe to a deposit or not.

1.2 default

Among 'default', 'yes' and 'no' were all selected. As we have seen in the data exploration process, there are only three people who have a value of 'yes' in the total train data. For this reason, we thought that such a small number of data was not important in analyzing the data and it is reasonable to drop this variable. However, as a result of feature importances, this variable was not dropped but the importance of 'default_0' and 'default_1' were all zero. We can interpret that although this 'default' variable was selected, it does not have much impact on predicting the 'y' value because the importance was zero.

1.3 contact

Among 'contact', 'cellular' and 'telephone' are both selected. Looking at Figure 15, the 'yes_ratio' of 'cellular' was higher than that of 'telephone'. Thus, we have already thought that both categories might be important in model prediction. Through the relatively high

degree of feature importance of ‘telephone’ as 62 and that of ‘cellular’ as 15, we concluded that our assumption from Figure 15 was quite correct.

1.4 month

As Figure 40 shows, among other months that have lower importance in predicting the model, May has the highest value of importance to model predicting. This is because, according to Figure 35, ‘may’ is the most frequent month that the contact is made up. It accounts for a large proportion of the total ‘month’, this means that the ‘may’ value can represent the entire bank deposit subscription, so this value could be considered important. Also, other values like ‘apr’, ‘jun’, ‘nov’, and ‘aug’ can also represent the data distribution that ‘may’ does not represent, so they are selected and used importantly in model prediction.

The attributes ‘mar’, ‘sep’, and ‘dec’ are the highest months that clients responded positively from bank deposits. As we estimated in ‘I. Data Exploration’, we thought that the reason why the proportion of subscribers is more than other months is because the clients want to start anew in the New Year. This effect is called the ‘Fresh Start Effect’, which means “a psychological phenomenon that signifies the propensity of individuals to take action towards their goals following specific temporal landmarks such as the start of a new week, month, or year”. Regarding the customers who tend to respond ‘yes’, the major features of them are, the job of themselves are ‘retired’ or ‘student’. Related to the ‘Fresh Start Effect’, ‘retired’ people who want to prepare for a stable financial future would subscribe to the deposit more. Also, the ‘student’ who meets people with a fresh mind in March and September, could show a high tendency to build a new goal, so they might join the deposit to achieve it.

1.5 day_of_week

Among ‘day of week’, ‘mon’ and ‘wed’ were selected. In Figure 40, the importance of these features is quite high. We interpreted that these two categories represent the entire ‘day of week’ variable.

1.6 campaign

'Campaign' has the second highest feature importance in model prediction. Looking at Figure 21, the 'yes_ratio' is higher when the number of performing campaigns is between '1' and '15'. It can be interpreted that conducting too many campaigns increases people's resistance, rather leading them not to subscribe. Based on this result, to attract the client's interest, making contacts with them without being too excessive. Therefore, we concluded that the number of performed campaigns is quite influential in predicting whether people will subscribe to a deposit or not.

1.7 poutcome

According to Figure 24, the proportion of 'nonexistent' values is large enough to affect the entire data prediction negatively. This is why the importance of 'poutcome_1('nonexistent')' is lowest among the 'poutcome' values. However, it is evident that the 'success' of the 'poutcome' attributes leads to a client subscribing to the bank deposit. Regarding this analysis, we can assume that the 'failure' of the 'poutcome' feature is as important as the 'success' value. This is why the importance of 'poutcome' attributes are calculated high, based on predicting the model.

1.8 Social and Economic context attributes

The problem we are solving now is the bank term deposit, so we can predict that the social and economic context attributes are mainly important values. This is why these features are considered important as a whole.

1.8.1 emp.var.rate

As mentioned above, 'emp.var.rate' indicates whether people are being employed steadily without any unstable fluctuations. Thus, we interpreted 'emp.var.rate' in the similar context of 'job'. Among job categories, easily replaceable categories like 'blue-collar' and 'housemaid' will be greatly affected by 'emp.var.rate', whereas specialized and irreplaceable categories like 'entrepreneur' and 'self-employed' will not be as impacted. Therefore,

‘emp.var.rate’ determines people’s economic stability, which in turn will significantly influence the prediction of their bank term deposit subscription prediction.

1.8.2 cons.price.idx

According to 1.17, the ‘cons.price.idx’ is highly related with market interest rate. The market interest rate, which is ‘euribor3m’ in this problem(refer to 1.8.4), and the consumer price index have to be compared and derived from the status of subscription. This is why the ‘cons.price.idx’ has a high importance among the many variables.

1.8.3 cons.conf.idx

As mentioned above, ‘cons.conf.idx’ quantifies the degree of confidence consumers feel about the economic situation. As the value of this variable fluctuates, it has a significant impact on consumers’ consumption and their decision to subscribe to a deposit. For this reason, we interpreted that the feature importance of ‘cons.conf.idx’ is very high.

1.8.4 euribor3m

The ‘euribor3m’ is the extremely most important factor of model prediction, according to Figure 40. As we can find out that this dataset is based on Portuguese, it is significant to use Euribor as a predicting feature. This is why we can assume the ‘euribor3m’ as the market interest of our problem. Based on this, we estimate the reason for this result. Regarding 1.8.2, ‘cons.price.idx’ is working by being compared to Euribor, actually the market interest of that time, and also, according to 1.8.3, ‘cons.conf.idx’ is based on the stability of the economic situation, which is highly correlated with the Euribor rate. The relationship between Euribor and the ‘emp.var.rate’ attribute is as similar as other economic and social features. Therefore, the ‘euribor3m’ could become the most important value in predicting the model.

2. Analysis of well-predicted data as class ‘1’

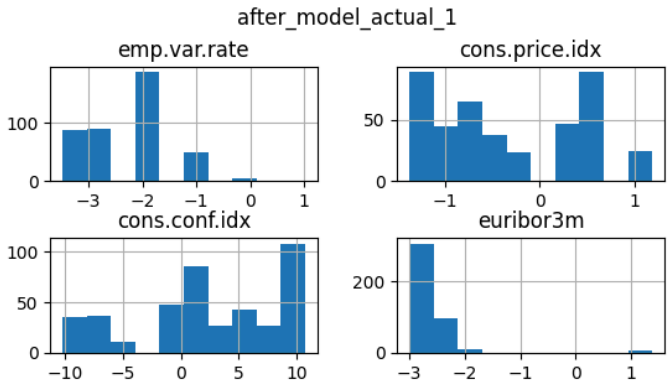


Figure 41: Distribution of numerical variables of data correctly predicted as a class '1'

After we got a model that yields good predictions and a considerably high f1 score, we thought that it is necessary to understand the economic characteristics of people who are predicted to be subscribed to the bank's term deposits using our model. It is because establishing a substantial foundation for developing practical strategies is important in the real world. We chose only numerical variables since all of them were selected in stepwise feature selection all the time. Also, they were all important and substantial standards for predicting people's actual economic situations. Thus, we plot a histogram which shows a distribution of 'emp.var.rate', 'cons.price.idx', 'cons.conf.idx', and 'euribor3m' of data correctly predicted as a class '1' as shown in Figure 41. Before beginning the analysis, since we normalized all values, scopes of values for each feature in Figure 41 are quite different from the original scopes. Therefore, we will use the range of the original data in the interpretation.

First, looking at Figure 41, actual subscribed clients' 'emp.var.rate' is highly distributed in the range of -3.5 to -2.5 and around -2. This means that actual subscribed clients have lower 'emp.var.rate', which indicates they have more stable jobs and income than others. This result seems to affirm the general thought that people with stable occupation are more likely to take a bank term deposit. Therefore, we can assume that there is the lack of creation of jobs in the labor market and it is another interpretation of the case of low 'emp.var.rate'. Therefore, this favorable condition of the labor market rather makes clients subscribe to the bank term deposits.

Second, actual subscribed clients' 'cons.price.idx' is highly distributed in the range of 92 to

93.5, which is to be the case of -1.5 to 0 in Figure 41. This means that actual subscribed clients have relatively low 'cons.price.idx', which implies that there is a decrease in prices coupled with an economic downturn. Eventually, these two factors lead people to subscribe to the bank term deposits.

Third, actual subscribed clients' 'cons.conf.idx' is highly distributed in the range of -41 to -30.5, which is to be the case of -0.5 to 10. This indicates that actual subscribed clients have relatively high 'cons.conf.idx'. It can be interpreted that most of the subscribed clients have a forward-looking and stable economic mindset.

Finally, actual subscribed clients' 'euribor3m' is highly distributed in the range of 0.5 to 1.5, which is to be the case of -3 to -2. This implies that actual subscribed clients have relatively low 'euribor3m'. Based on distribution of 'cons.conf.idx' and 'euribor3m', we could conclude that most of the subscribers tend to avoid risks as much as possible and plan for an economically secure future. Thus, the bank should target clients who prepare for their future financial situation and those who are planning-oriented.

Up to this point, we have analyzed four economic indicators that strongly influence the model's predictions. Through this analysis, we gained socio-economic insights from each of them. Additionally, we discovered that as indicated in 'emp.var.rate' results, we should consider societal context's impacts on the economy. Also, we should never overlook the individual clients' tendencies, reflected in the results of 'cons.conf.idx' and 'euribor3m'.

V. References

https://github.com/bayesian-optimization/BayesianOptimization

https://www.larksuite.com/en_us/topics/productivity-glossary/fresh-start-effect

https://medium.com/@asanjyal81/dimensionality-reduction-vs-feature-selection-e68f91aa8724

https://ok-lab.tistory.com/191

https://medium.com/@VitorCSampaio/understanding-ordinary-least-squares-ols-the-foundation-of-linear-regression-1d79bfc3ca35

https://scikit-learn.org/stable/